Phần 1: Phân tích & Thẩm định mã nguồn (Code Auditing)

1. Giải thích các tham số trong câu lệnh nguy hiểm

Câu lệnh của AI: `Remove-Item -Path "C:\*" -Recurse -Force`

- **`-Path "C:\*"`:** Chỉ định mục tiêu xóa là toàn bộ mọi tệp tin, thư mục con nằm ngay tại gốc của ổ đĩa hệ thống (`C:\`). Đây là nơi lưu trữ nhân hệ điều hành Windows (`C:\Windows`), dữ liệu người dùng (`C:\Users`), và các tệp cài đặt chương trình (`C:\Program Files`).
- **`-Recurse`:** Đệ quy toàn bộ cây thư mục. Lệnh sẽ không dừng lại ở thư mục gốc mà chui sâu vào từng cấp thư mục con bên trong để quét sạch mọi thứ.
- **`-Force`:** Bỏ qua các cơ chế bảo vệ thông thường của hệ điều hành. Tham số này ép buộc xóa cả các tệp có thuộc tính ẩn (`Hidden`) hoặc chỉ đọc (`ReadOnly`) mà không cần hỏi xác nhận từ người dùng.

2. Hậu quả khi thực thi

- **Sập toàn bộ hệ điều hành (BSOD / Unbootable System):** Hệ điều hành sẽ cố gắng xóa các tệp DLL, Registry, driver và tệp khởi động hệ thống. Kết quả là máy chủ bị treo ngay lập tức, crash sang màn hình xanh và không thể khởi động lại (phải cài lại toàn bộ OS).
- **Mất dữ liệu toàn diện không thể phục hồi:** Cơ chế của `Remove-Item` trên dòng lệnh không chuyển tệp vào Thùng rác (Recycle Bin) mà xóa vĩnh viễn khỏi bảng phân bổ tệp (MFT). Toàn bộ mã nguồn, cơ sở dữ liệu, và nhật ký của "Trang trại Lan thông minh" sẽ biến mất hoàn toàn.

3. Đối chiếu 3 Nguyên tắc "Vàng" thẩm định mã nguồn do AI gợi ý

1. **Nguyên tắc 1: Không chạy mã nguồn khi chưa hiểu rõ từng cờ/tham số (Understand before Run):** Phải phân tích rõ ràng từng tham số (`-Path`, `-Recurse`, `-Force`). Việc copy-paste mù quáng mà không hiểu cờ `-Recurse` trên thư mục gốc tương đương với lệnh tự hủy hệ thống `rm -rf /*` trên Linux.
2. **Nguyên tắc 2: Khoanh vùng đường dẫn tuyệt đối an toàn (Strict Scoping & Least Privilege):** Tuyệt đối không bao giờ cho phép các lệnh mang tính phá hủy (xóa, format) hoạt động trên các đường dẫn cấp hệ thống (`C:\`, `C:\Windows`) hoặc ký tự đại diện không kiểm soát (*). Phạm vi phải luôn được giới hạn chặt chẽ bên trong thư mục con của dự án (ví dụ: `C:\SmartFarm\logs\`).
3. **Nguyên tắc 3: Luôn chạy thử ở chế độ mô phỏng (`-WhatIf` / Dry-run):** Trước khi chạy bất kỳ lệnh `Remove-Item` nào, phải thêm tham số `-WhatIf` để

PowerShell liệt kê danh sách tệp sẽ bị tác động mà chưa thực sự xóa bất cứ thứ gì.

Phần 2: Triển khai kịch bản an toàn (`safe_cleanup.ps1`)

Kịch bản được viết lại tuân thủ nguyên tắc khoanh vùng đường dẫn và xử lý đúng yêu cầu:

PowerShell

```
# ================================================================================
# Script Name : safe_cleanup.ps1
# Description : Loc log can chu y va don dep cac tep tam (.tmp) an toan
# ================================================================================

# Dừng ngay lập tức nếu gặp bất kỳ lỗi nghiêm trọng nào
$ErrorActionPreference = "Stop"

# Khai báo đường dẫn thư mục dự án
$ProjectLogsDir = "C:\SmartFarm\logs"

Write-Host "[1/3] Dang di chuyen den thu muc logs cua du an..." -ForegroundColor Cyan
if (Test-Path -Path $ProjectLogsDir) {
    Set-Location -Path $ProjectLogsDir
} else {
    Write-Host "[ERROR] Thu muc khong ton tai: $ProjectLogsDir" -ForegroundColor Red
    exit 1
}

Write-Host "[2/3] Dang loc cac dong WARNING tu system.log sang issues.txt..." -ForegroundColor Cyan
if (Test-Path -Path ".\system.log") {
    # Lọc nội dung WARNING và lưu vào issues.txt
    Select-String -Path ".\system.log" -Pattern "WARNING" | Out-File -FilePath ".\issues.txt" -Encoding utf8
    Write-Host "      -> Da tao thanh cong tep issues.txt" -ForegroundColor Green
```

```
} else {
    Write-Host "      -> [SKIP] Khong tim thay tep system.log de
loc" -ForegroundColor Yellow
}

Write-Host "[3/3] Dang don dep cac tep tam (.tmp) trong thu muc
logs..." -ForegroundColor Cyan
# Áp dụng Nguyên tắc Khoanh vùng: Chỉ xóa tệp có đuôi .tmp nằm bên
trong thư mục hiện tại
Remove-Item -Path ".\*.tmp" -Force -ErrorAction SilentlyContinue

Write-Host "[SUCCESS] Quy trinh bao tri an toan hoan tat!" -
ForegroundColor Green
```

Phần 3: Quy trình kiểm thử & Xác minh (Verify)

Sau khi chạy `safe_cleanup.ps1`, thực hiện quy trình kiểm tra 3 bước trực tiếp trên PowerShell để xác nhận:

1. Kiểm tra sự tồn tại của tệp báo cáo với `Test-Path`

Xác nhận tệp `issues.txt` đã được khởi tạo thành công:

PowerShell
```
Test-Path -Path ".\issues.txt"
```

- **Kỳ vọng:** Trả về `True`.

2. Xem nội dung dữ liệu lọc được với `Get-Content`

Đảm bảo các dòng log được trích xuất chính xác là cảnh báo `WARNING`:

PowerShell
```
Get-Content -Path ".\issues.txt" -TotalCount 10
```

- **Kỳ vọng:** Terminal in ra tối đa 10 dòng đầu tiên của `issues.txt`, mỗi dòng đều chứa chuỗi khớp với mẫu `WARNING`.

3. Xác nhận tệp `.tmp` đã được dọn sạch với `Get-ChildItem`

Kiểm tra xem còn sót tệp rác `.tmp` nào trong thư mục hay không:

PowerShell
```
Get-ChildItem -Path ".\*.tmp"
```

- **Kỳ vọng:** Không có tệp nào được trả về (Terminal trống hoặc thông báo không tìm thấy đường dẫn khớp), chứng minh toàn bộ các tệp `.tmp` đã được xóa sạch sẽ mà không ảnh hưởng đến các tệp log chính (`system.log`, `issues.txt`).